Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 05 · Part B: Mathematics of Signals and Systems Level 2

Track: Laplace Transform, Transfer Functions, Pole-Zero Analysis, Stability, Bode Plots, Attitude Control

Prerequisites: Problems 03, 04 · **Estimated effort:** 10–14 hours

Laplace Transform, Poles and Stability: Reading a System from Its Poles

The location of a handful of complex numbers, the poles, determines whether a system rings, settles, or runs away. Compute Laplace transforms by hand and symbolically, link pole positions to time responses, derive the step-response formulas of the second-order system, test stability with the Routh–Hurwitz criterion, draw Bode plots, and design a PD controller for a satellite.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/05_laplace_transform_and_stability/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random seeds, (ii) a short written report (`report.pdf` or `report.md`) containing the requested analysis

sections, and (iii) all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

Fourier analysis describes steady-state sinusoidal behaviour; the Laplace transform adds the *transient*: it covers growth and decay as well as oscillation, handles initial conditions, and turns linear differential equations into algebra. A satellite attitude loop, an RC filter and an op-amp circuit are all rational functions of s, and the design questions (is it stable? how fast? how much overshoot?) become questions about where the poles lie.

This problem develops the toolbox a signals engineer needs before designing analog filters (problem 08) and digital filters (problems 14–15): transform pairs, pole-zero reading, standard second-order formulas, the Routh test, and Bode plots, with an attitude-control example from aerospace.

Theoretical Framework

1. Laplace Transform and ROC

For a causal signal, $X(s) = \int_{0^-}^{\infty} x(t)\mathrm{e}^{-st}\mathrm{d}t$ with $s = \sigma + \mathrm{j}\omega$. Useful pairs: $\delta(t) \to 1$; $u(t) \to 1/s$; $\mathrm{e}^{-at}u(t) \to 1/(s+a)$ with region of convergence (ROC) $\mathrm{Re}\, s > -a$; $t\mathrm{e}^{-at}u(t) \to 1/(s+a)^2$; $\sin(\omega_0 t)u(t) \to \omega_0/(s^2+\omega_0^2)$. Differentiation obeys $\dot{x} \to sX(s) - x(0^-)$; convolution becomes $Y = HX$. The same $X(s)$ with a different ROC is a different (for example anti-causal) signal, so the ROC is part of the answer. The Fourier transform is $X(s)$ on the imaginary axis if it lies in the ROC. Initial and final value theorems: $x(0^+) = \lim_{s\to\infty} sX$, $x(\infty) = \lim_{s\to 0} sX$ (if the latter exists).

2. Poles, Zeros and Stability

For a rational $H(s) = K\prod(s-z_i)/\prod(s-p_i)$ a pole at $p = \sigma + \mathrm{j}\omega$ contributes a term $\mathrm{e}^{\sigma t}\cos(\omega t + \phi)$ to the impulse response. A causal LTI system is BIBO stable if and only if all poles lie in the open left half-plane. The Routh–Hurwitz array tests this without computing roots: for $a_n s^n + \cdots + a_0$, build rows from the coefficients; the number of sign changes in the first column equals the number of right-half-plane roots.

3. Second-Order System

$H(s) = \omega_n^2/(s^2+2\zeta\omega_n s+\omega_n^2)$ has poles $s = -\zeta\omega_n \pm \mathrm{j}\omega_n\sqrt{1-\zeta^2}$ and, for $0 < \zeta < 1$, the step response metrics

$$M_p = \exp\left(-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}\right), \qquad t_p = \frac{\pi}{\omega_d}, \qquad t_s^{(2\%)} \approx \frac{4}{\zeta\omega_n}, \tag{1}$$

and the frequency response peaks at $\omega_r = \omega_n\sqrt{1-2\zeta^2}$ with $M_r = 1/(2\zeta\sqrt{1-\zeta^2})$ for $\zeta < 1/\sqrt{2}$.

4. Bode Plots

Evaluate $H(\mathrm{j}\omega)$ and plot $20\log_{10}|H|$ and $\angle H$ against $\log\omega$. Each real pole contributes $-20\,\mathrm{dB}$ per decade beyond its corner and $-45°$ phase at the corner; each zero the opposite. The $-3\,\mathrm{dB}$ bandwidth of a first-order low-pass $1/(1+s/\omega_c)$ is ω_c.

5. Satellite Attitude Control

A rigid satellite about one axis obeys $J\ddot{\theta} = \tau$, i.e. $P(s) = 1/(Js^2)$ (two poles at the origin: marginally unstable). A proportional–derivative controller $\tau = K_p(\theta_c - \theta) - K_d\dot{\theta}$ gives the closed loop

$$\frac{\theta(s)}{\theta_c(s)} = \frac{K_p}{Js^2 + K_d s + K_p}, \qquad \omega_n = \sqrt{K_p/J}, \quad \zeta = \frac{K_d}{2\sqrt{K_p J}}. \tag{2}$$

Data and Simulation Specification

Systems	$H_1 = 1/(s^2+3s+2)$, $H_2 = 1/(s^2+2s+5)$, $H_3 = 1/(s^2-s+2)$, $H_4 = 1/(s^2+0.2s+9)$, $H_5 = (s+1)/(s^2+s+6)$.
Characteristic equation	$s^3+6s^2+11s+6+K = 0$ for the Routh task (loop $L(s) = K/((s+1)(s+2)(s+3))$).
Satellite	Moment of inertia $J = 10\,\mathrm{kg}\cdot\mathrm{m}^2$; target $\omega_n = 0.3\,\mathrm{rad/s}$, $\zeta = 0.7$; 30° slew; measurement delay swept from 0 to 2 s.
Tools	`scipy.signal` (`step`, `impulse`, `bode`, `residue`, `tf2zpk`), SymPy for symbolic transforms.
Real data	The ruler-twang accelerometer record of problem 03 (or a swinging pendulum filmed or logged with Phyphox): fit ω_n and ζ and place the poles in the s-plane.

Your Task

1. **Transforms by hand and symbolically (Stage 2).** Derive the Laplace transforms of the pairs in Section 1 by integration, then confirm them with SymPy (`laplace_transform`). Invert $H_1(s)$ and $H_5(s)$ by partial fractions by hand, check with `scipy.signal.residue`, and compare the resulting impulse response with a numerical one.
2. **Poles and responses.** For $H_1, \ldots, H_5$ plot the pole-zero map and the impulse and step responses side by side. State, for each, the decay rate, oscillation frequency and stability. Demonstrate that the same $X(s) = 1/(s+1)$ with ROC $\mathrm{Re}\,s > -1$ and $\mathrm{Re}\,s < -1$ corresponds to $\mathrm{e}^{-t}u(t)$ and $-\mathrm{e}^{-t}u(-t)$.
3. **Second-order metrics.** Derive Eq. (1) for M_p and t_p. For ζ from 0.05 to 0.99 extract the overshoot and the 2% settling time numerically from `scipy.signal.step`, and plot them with the formulas. Draw the region of the s-plane that satisfies $M_p < 10\%$ and $t_s < 4\,\mathrm{s}$.

4. **Routh–Hurwitz.** Implement the Routh array as a function. Test it on random polynomials against `numpy.roots`. For the third-order characteristic equation find the range of K for stability, the critical gain and the crossing frequency, and verify with a root-locus sweep.
5. **Bode plots.** Draw the asymptotic Bode plot of H_4 and of a cascade of two first-order stages by hand; compare with `scipy.signal.bode`. Verify ω_r and M_r for $\zeta = 0.3$ and show that the $-3\,\mathrm{dB}$ point of a first-order system is at its pole.
6. **Satellite attitude controller.** Choose K_p and K_d for the target ω_n, ζ using Eq. (2), simulate the 30° slew, and report overshoot and settling time. Model a measurement delay with a first-order Padé approximation $\mathrm{e}^{-sT_d} \approx (1 - sT_d/2)/(1 + sT_d/2)$, find the pole locations as the delay grows and the delay at which the loop goes unstable.
7. **Identification.** Fit ω_n and ζ of your measured ringing record with a nonlinear least-squares fit of Eq. (1)'s impulse response, compare with the logarithmic-decrement estimate from problem 03 and put the two poles in the s-plane with error bars.

Self-check targets

Overshoot is 4.6% for $\zeta = 0.7$ and 16.3% for $\zeta = 0.5$. For $s^3+6s^2+11s+6+K$ the system is stable for $-6 < K < 60$, the critical gain is $K = 60$ and the crossing frequency is $\omega = \sqrt{11} = 3.317\,\mathrm{rad/s}$. For $\zeta = 0.3$ the resonant peak is $M_r = 1.747$ (4.85 dB). The attitude gains for the stated design are $K_p = J\omega_n^2 = 0.9\,\mathrm{N\,m/rad}$ and $K_d = 2\zeta\omega_n J = 4.2\,\mathrm{N\,m\,s/rad}$.

Aerospace Application Link

Star trackers and gyros deliver attitude with a processing delay. With your Task 6 model, compute how much latency the $\zeta = 0.7$ attitude loop tolerates before its damping falls below 0.3 and before it goes unstable, and express it as a fraction of the closed-loop time constant $1/(\zeta\omega_n)$. State the maximum star-tracker latency (in seconds) that the design can accept, and what you would change (bandwidth or damping) if the tracker's latency were doubled.

Deliverables

D1 **Transform notebook:** hand derivations, SymPy confirmations and partial-fraction inversions with numerical checks.

D2 **Pole-zero gallery:** the five systems with maps, responses and written interpretation, plus the ROC demonstration.

D3 **Second-order and Routh study:** overshoot and settling curves versus ζ with the s-plane specification region; the Routh function with tests and the critical-gain result.

D4 **Bode study:** hand-drawn asymptotes next to computed plots, resonance check.

D5 **Attitude controller:** the gains, slew simulation, delay-margin analysis and the system identification of the real ringing record.

D6 **Memo (250 words):** "How much sensor latency can the attitude loop tolerate?" Cite at least two external sources.

Bonus (a) Plot the root locus of $L(s) = K/(s(s+2)(s+5))$ by hand rules and compare with `python-control`. (b) Derive the Laplace-domain model of an RLC circuit from its differential equation and confirm it by measurement. (c) Add torque saturation and show the integrator wind-up problem with an integral term.

Learning Resources

Laplace Transform and Systems

- Oppenheim, A.V., Willsky, A.S. *Signals and Systems*, Ch. 9 (Laplace transform).
- Lathi, B.P. *Linear Systems and Signals*, chapters on the Laplace transform and system response.
- Nise, N.S. *Control Systems Engineering*, Ch. 2–4 (modelling, time response) and Ch. 6 (stability, Routh–Hurwitz).

Attitude Control

- Wie, B. *Space Vehicle Dynamics and Control* (single-axis attitude control).
- Wertz, J.R. (ed.) *Spacecraft Attitude Determination and Control* (sensors and their latency).

Tools

- SciPy signal, SymPy: `https://docs.scipy.org/doc/scipy/reference/signal.html`, `https://docs.sympy.org`
- Python Control Systems Library (optional): `https://python-control.readthedocs.io`

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.